

LF5-Grundlagen: Python-Übungen

1. Notenberechnung aus Punkten

Ziel: Punkte (0-100) in Noten (1-5) umwandeln mit if/elif/else.

```
punkte = int(input("Geben Sie die Punkte 0 bis 100 ein: "))
if punkte < 0 or punkte > 100:
    print("Ungültige Eingabe. Bitte gib eine Zahl zwischen 0 und 100 ein!")
elif punkte >= 90:
    print("Sie haben die Note 1, Sehr gut")
elif punkte >= 80:
    print("Sie haben die Note 2, Gut")
elif punkte >= 70:
    print("Sie haben die Note 3, Befriedigend")
elif punkte >= 60:
    print("Sie haben die Note 4, Ausreichend")
else:
    print("Note 5, Sie haben nicht bestanden!")
```

Lernziele: input(), int(), Vergleichsoperatoren, verschachtelte if-Bedingungen.

2. Passwortprüfung (3 Versuche)

Ziel: while-Schleife mit Zähler und break.

```
passwort = "pass"
z = 0
while z < 3:
    z += 1
    eingabe = input("Geben Sie das Passwort ein: ")
    if eingabe == passwort:
        print("Zugriff gewährt!")
        break
    elif z != 3:
        print(f"Falsches Passwort! Noch {3-z} Versuche")
    else:
        print("Zugriff verweigert!")
```

Lernziele: while-Schleife, Zähler, break, f-Strings.

3. Teiler einer Zahl finden

Ziel: for-Schleife + math.sqrt für Teilerpaare.

```
import math
eingabe = int(input("Geben Sie die Zahl ein: "))
print(f"Teiler der Zahl {eingabe} sind:")
for i in range(1, int(math.sqrt(eingabe)) + 1):
    if eingabe % i == 0:
        print(f"{i} --Komplementrteiler-- {eingabe//i}")
```

Lernziele: import math, range(), Modulo (%), Ganzzahldivision (/).

4. Vollständige Begrüßung

Ziel: Mehrere input()-Aufrufe kombinieren.

```
vorname = input("Gib deinen Vornamen ein: ")
nachname = input("Gib deinen Nachnamen ein: ")
print(f"Guten Tag {vorname} {nachname}")
```

Lernziele: String-Konkatenation mit f-Strings, mehrere Variablen.

Lernreihenfolge

1. Notenberechnung → verstehe if/elif/else
2. Passwort → verstehe while + break
3. Teiler → verstehe for + math.sqrt
4. Begrüßung → verstehe input() kombinieren

Nächster Schritt: Schülernoten-Listen (append, min/max/sum)

LF5-Grundlagen: Python-Übungen (Teil 2)

5. Schülernoten-Listen (Basis)

Ziel: Listen mit for-Schleife füllen, min/max/sum berechnen.

```
anzahl_schueler = int(input("Wieviel Schüler wurden benotet? "))
noten = []
for i in range(1, anzahl_schueler + 1):
    note = int(input(f"Geben sie die Note für Schüler {i} ein: "))
    noten.append(note)

print("Alle Noten:", noten)
print("Beste Note:", min(noten))
```

```
print("Schlechteste Note:", max(noten))
summe = sum(noten)
durchschnitt = summe / anzahl_schueler
print("Durchschnitt:", round(durchschnitt, 2))
```

Lernziele: `append()`, `min()`/`max()`/`sum()`, Listen-Indexierung.

6. Einkaufsliste mit Zählung

Ziel: while-Schleife für Input, Duplikate zählen.

```
einkaufsliste = []
artikel = input("tippe 'fertig' um zu beenden, Neuer Artikel: ")
while artikel != "fertig":
    einkaufsliste.append(artikel)
    artikel = input("tippe 'fertig' um zu beenden, Neuer Artikel: ")
einkaufsliste.remove("fertig")

print("Einkaufsliste:", einkaufsliste)

# Einzigartige Artikel + Zählung
einzigartige_liste = []
for element in einkaufsliste:
    if element not in einzigartige_liste:
        einzigartige_liste.append(element)

print("Einzigartige Artikel:", einzigartige_liste)
for produkt in einzigartige_liste:
    count = 0
    for i in einkaufsliste:
        if produkt == i:
            count += 1
    print(f"{produkt}: x{count}")
```

Lernziele: Nested Loops, `in`-Operator, Listenzählung.

7. Vier-Rechenarten-Rechner

Ziel: if/elif für Operationen, Division durch 0 prüfen.

```
zahl1 = int(input("Gib die erste Zahl ein: "))
zahl2 = int(input("Gib die zweite Zahl ein: "))

addition = zahl1 + zahl2
subtraktion = zahl1 - zahl2
multiplikation = zahl1 * zahl2
```

```

if zahl2 != 0:
    division = zahl1 / zahl2
    print(f"Die Summe der Zahlen {zahl1} und {zahl2} ist {addition}")
    print(f"Die Differenz der Zahlen {zahl1} und {zahl2} ist {subtraktion}")
    print(f"Das Produkt der Zahlen {zahl1} und {zahl2} ist {multiplikation}")
    print(f"Der Quotient der Zahlen {zahl1} und {zahl2} ist {division}")
else:
    print(f"Die Summe der Zahlen {zahl1} und {zahl2} ist {addition}")
    print(f"Die Differenz der Zahlen {zahl1} und {zahl2} ist {subtraktion}")
    print(f"Das Produkt der Zahlen {zahl1} und {zahl2} ist {multiplikation}")
    print("Die Division durch 0 ist nicht erlaubt.")

```

Lernziele: Mathematische Operatoren, Zero-Division-Prüfung.

8. Sternchen-Triangel (auf/ab)

Ziel: range() mit verschiedenen Schritten.

```

anzahl = int(input("Geben Sie die maximale Anzahl an Sternchen in einer Linie an:
"))

# Aufbau (1,3,5,... Sternchen)
for i in range(1, anzahl + 1):
    print(" " * (anzahl - i) + "*" * (2 * i - 1))

# Abbau
for i in range(anzahl - 1, 0, -1):
    print(" " * (anzahl - i) + "*" * (2 * i - 1))

print(f"Hinweis: Es sind insgesamt {anzahl * (anzahl + 1)} Sternchen")

```

Lernziele: range(start, stop, step), String-Multiplikation *.

Lernreihenfolge (Fortsetzung)

5. Schülernoten → verstehe append/min/max/sum
6. Einkaufsliste → verstehe nested loops + count
7. Rechner → verstehe alle Rechenoperatoren
8. Triangel → verstehe range() Variationen

Nächster Schritt: Statistics-Modul + Funktionen (Teil 3)

LF5-Grundlagen: Python-Übungen (Teil 3)

9. Statistics-Modul (Erweiterte Analysen)

Ziel: import statistics für professionelle Listen-Analyse.

```
import statistics

anzahl_schueler = int(input("Wieviel Schüler wurden benotet? "))
noten = []
for i in range(1, anzahl_schueler + 1):
    note = int(input(f"Geben sie die Note für Schüler {i} ein: "))
    noten.append(note)

print("Alle Noten:", noten)
print("Beste Note:", min(noten))
print("Schlechteste Note:", max(noten))
print("Durchschnitt:", round(statistics.mean(noten), 2))

print(f"{statistics.median(noten)} (Median)")
print(f"{statistics.variance(noten)} (Varianz)")
print(f"{statistics.stdev(noten)} (Standardabweichung)")
print(f"{statistics.mode(noten)} (Modalwert)")
```

Lernziele: import statistics, mean/median/variance/stdev/mode.

10. Kehrwert-Funktion mit Exception-Handling

Ziel: try/except für sichere Division.

```
def kehrwert_berechnen():
    try:
        zahl = int(input("Gib eine ganze Zahl ungleich null ein: "))
        kehrwert = 1 / zahl
        print(f"Der Kehrwert ist 1/{zahl} = {kehrwert}")
    except ValueError as e:
        print("Es ist ein ValueError aufgetreten. Gib nächstes mal eine ganze Zahl ein!")
    except ZeroDivisionError as e:
        print("ZeroDivisionError ist aufgetreten. Null ist nicht erlaubt!")

kehrwert_berechnen()
```

Lernziele: Funktionen (def), try/except, ValueError/ZeroDivisionError.

11. Taschenrechner mit Funktionen

Ziel: Separate Funktionen für jede Operation.

```

def addition(a, b):
    ergebnis = a + b
    return ergebnis

def subtraktion(a, b):
    ergebnis = a - b
    return ergebnis

def multiplikation(a, b):
    ergebnis = a * b
    return ergebnis

def division(a, b):
    if b != 0:
        ergebnis = a / b
        return ergebnis
    else:
        print("Division durch 0 nicht möglich")
        return None

x = float(input("Gib die erste Zahl ein: "))
y = float(input("Gib die zweite Zahl ein: "))
z = input("Gib die Rechenoperation ein (+,-,*,/): ")

if z == "+":
    print(addition(x, y))
elif z == "-":
    print(subtraktion(x, y))
elif z == "*":
    print(multiplikation(x, y))
elif z == "/":
    print(division(x, y))
else:
    print("Ungültige Eingabe")

```

Lernziele: `return`, Parameter, `float()` für Kommazahlen.

12. Temperaturumrechnung

Ziel: Funktionen mit `if/elif` für Einheiten.

```

def convert_temperature(temp, einheit):
    if einheit == "C":
        temp_f = temp * 9/5 + 32
        return f"{temp_f}°F"
    elif einheit == "F":
        temp_c = (temp - 32) * 5/9
        return f"{temp_c}°C"
    else:
        return "Falsche Eingabe!!"

```

```
temperatur = float(input("Geben Sie die Temperatur an: "))
einheit1 = input("Geben Sie die Einheit an (F für Fahrenheit, C für Celsius): ")
print(convert_temperature(temperatur, einheit1))
```

Lernziele: Funktionen mit mehreren return-Werten, Kommazahlen-Rechnung.

Lernreihenfolge (Fortsetzung)

```
9. Statistics → verstehe Statistik-Funktionen
10. Kehrwert → verstehe try/except + def
11. Rechner-Funktionen → verstehe return + Parameter
12. Temperatur → verstehe komplexe Funktionen
```

Nächster Schritt: Datei-Behandlung + Listen-Operationen (Teil 4)

LF5-Grundlagen: Python-Übungen (Teil 4)

13. Datei lesen/schreiben (Längste Zeile)

Ziel: Datei-Operationen mit try/except.

```
dat = input("Geben Sie den Namen der einzulesenden Datei ohne Endung ein: ")
try:
    file = open(f"{dat}.txt", "r", encoding="utf-8")
    text = file.readlines()
    liste = []
    for i in range(10):
        print(text[i])
        liste.append(text[i])
    file.close()

    laengste_zeile = ""
    for i in liste:
        if len(i) > len(laengste_zeile):
            laengste_zeile = i

    file2 = open("dat3.txt", "w", encoding="utf-8")
    file2.write(laengste_zeile)
    file2.close()

except IOError as e:
    print("Fehler!")
    print(e)
except Exception as e:
```

```
print("Auch Fehler")
print(e)
```

Lernziele: `open()`, `readlines()`, `write()`, IOError-Handling.

14. Listen-Operationen (sort/pop/insert)

Ziel: Alle wichtigen List-Methoden.

```
zahlen = [3, 7, 2, 9, 4, 1]
print(zahlen[0])      # Erstes Element
print(zahlen[5])      # Letztes Element

zahlen.sort()
print(zahlen[0], zahlen[-1]) # Min/Max nach Sortierung

for zahl in zahlen:
    if zahl > 20:
        print(zahl)

zahlen.clear()
hilfsliste = [2, 0, 55, 2, 3, 4, 11]
for i in hilfsliste:
    zahlen.append(i)

print(zahlen)
print(zahlen[0], zahlen[len(zahlen)-1])
```

Lernziele: `sort()`, `clear()`, `pop()`, `insert()`, Index -1.

15. Sternchen-Quadrat

Ziel: Nested for-Schleifen für Muster.

```
anzahl = int(input("Geben Sie die maximale Anzahl an Sternchen in einer Linie an: "))

# Quadrat mit Sternchen
for i in range(anzahl):
    print("*" * anzahl)

print(f"Hinweis: Es sind insgesamt {anzahl * anzahl} Sternchen")
```

Lernziele: Nested Loops, String-Multiplikation.

16. Backslash-Ausgabe

Ziel: Escape-Sequenzen verstehen.

```
print("Ausgabe EINES! Backslashes: \\")
print("Ausgabe zweier Backslashes: \\\\")
print("Ausgabe auch mit Leerzeichen möglich: \\ \\")
```

Lernziele: \\ für einzelnen Backslash, Escape-Sequenzen.

Lernreihenfolge (Fortsetzung)

```
13. Dateien → verstehe open/read/write
14. Listen-Methoden → sort/clear/append/pop
15. Sternchen-Quadrat → nested loops
16. Backslash → Escape-Sequenzen
```

Nächster Schritt: Funktionen erweitern + Zufallszahlen (Teil 5)

LF5-Grundlagen: Python-Übungen (Teil 5)

17. Zufallszahl-Raten (try/except)

Ziel: random + Exception-Handling kombinieren.

```
import random

zufallszahl = random.randint(1, 10)
try:
    eingabe = int(input("Treffen Sie die Zahl? Geben Sie eine ganze Zahl von 1 bis 10 ein: "))
    if eingabe < 1 or eingabe > 10:
        print("Ungültige Eingabe!")
    elif zufallszahl == eingabe:
        print("Herzlichen Glückwunsch! Sie haben gewonnen!")
    else:
        print(f"Sie haben leider verloren! Zum Vergleich Ihre Eingabe {eingabe}, aber die gesuchte Zahl war {zufallszahl}")
except ValueError:
    print("Die Aufgabe war: Geben Sie eine ganze Zahl von 1 bis 10 ein!!!!!!")
```

Lernziele: import random, randint(), try/except ValueError.

18. Primzahlen prüfen

Ziel: math.sqrt + nested Loops für Primzahlen.

```
import math

x = int(input("Geben Sie eine Zahl ≥ 1 ein: "))
ist_primzahl = True
for i in range(2, int(math.sqrt(x)) + 1):
    if x % i == 0:
        ist_primzahl = False
        break

if ist_primzahl:
    print(f"{x} ist eine Primzahl!")
else:
    print(f"{x} ist keine Primzahl!")
```

Lernziele: Primzahlerkennung, Optimierung mit sqrt.

19. Palindrom prüfen

Ziel: String rückwärts vergleichen.

```
x = input("Geben Sie ein Wort ein: ")
y = x.lower()
is_palindrome = True
for i in range(len(x) // 2):
    if y[i] != y[-1 - i]:
        is_palindrome = False
        break

if is_palindrome:
    print("Palindrom!")
else:
    print("Kein Palindrom!")
```

Lernziele: String-Indexierung, lower(), Negativ-Index.

20. Basis-Umrechnung (Dezimal → Basis b)

Ziel: While-Schleife für Zahlensysteme.

```
def dezimal_zu_basis(zahl, basis):
    zeichen = "0123456789ABCDEFGHIJKLMNOPQRSTUVWXYZ"
    if zahl < 0:
```

```

        return "0"
    basiszahl = ""
    while zahl > 0:
        rest = zahl % basis
        basiszahl = zeichen[rest] + basiszahl
        zahl = zahl // basis
    return basiszahl

b = int(input("Geben Sie die Basis (2-36) ein: "))
while b < 2 or b > 36:
    b = int(input("Ungültige Basis! Bitte geben Sie eine Basis zwischen 2 und 36 ein: "))

dezimalzahl = int(input("Geben Sie eine Dezimalzahl ein: "))
print(f"Die Dezimalzahl {dezimalzahl} zur Basis {b} ist: {dezimal_zu_basis(dezimalzahl, b)}")

```

Lernziele: Rekursive Division, String-Bau vorne anhängen.

Lernreihenfolge (Fortsetzung)

```

17. Zufallszahl → random + try/except
18. Primzahlen → math.sqrt + nested loops
19. Palindrom → String-Indexierung
20. Basis-Umrechnung → while + Modulo

```

Nächster Schritt: Klassen + Objektorientierung (Teil 6)

LF5-Grundlagen: Python-Übungen (Teil 6)

21. Klasse Musikinstrument (OOP Grundlagen)

Ziel: Erste Klasse mit Konstruktor und Methoden.

```

class Musikinstrument:
    def __init__(self, modell="unbekannt", hersteller="unbekannt", preis=0.0):
        self.modell = modell
        self.hersteller = hersteller
        self.preis = preis

    def set_modell(self, value):
        self.modell = value

    def get_modell(self):
        return self.modell

```

```

    def get_daten(self):
        daten = f"Modell: {self.modell}, Hersteller: {self.hersteller}, Preis: {self.preis}"
        return daten

# Objekt erstellen und verwenden
instrument = Musikinstrument("PRS Custom 24 BW", "PRS", 2500.0)
print(instrument.get_daten())

instrument.set_modell("PRS Spezial")
print(instrument.get_modell())

```

Lernziele: `class`, `__init__()`, `self`, Getter/Setter-Methoden.

22. Subklasse Gitarre

Ziel: Vererbung mit `super()`.

```

class Gitarre(Musikinstrument):
    def __init__(self, modell="unbekannt", hersteller="unbekannt", preis=0.0, groesse="unbekannt"):
        super().__init__(modell, hersteller, preis) # Superklasse aufrufen
        self.groesse = groesse

    def set_groesse(self, value):
        self.groesse = value

    def get_groesse(self):
        return self.groesse

# Gitarre-Objekt erstellen
gitarre = Gitarre("Fender Stratocaster", "Fender", 800.0, "mittel")
print(gitarre.get_daten())
print("Größe:", gitarre.get_groesse())

```

Lernziele: Vererbung (`super()`), erweiterte Klassen.

23. GGT/KGV (Euclid-Algorithmus)

Ziel: Rekursiver Algorithmus mit `while`.

```

a = int(input("Geben Sie die erste Zahl ein: "))
b = int(input("Geben Sie die zweite Zahl ein: "))

temp1 = a
temp2 = b
while temp2 != 0:

```

```

if temp1 > temp2:
    temp1 = temp1 - temp2
else:
    temp2 = temp2 - temp1

print(f"Der ggT (größter gemeinsamer Teiler) von {a} und {b} ist {temp1}")
print(f"Das kgV (kleinstes gemeinsames Vielfache) von {a} und {b} ist {a*b//temp1}")

```

Lernziele: Euclid-Algorithmus, Ganzzahldivision //.

24. Fibonacci-Folge

Ziel: Listen mit for-Schleife füllen.

```

n = int(input("Geben sie den oberen Index an: "))
fibonacci = [0, 1]
for i in range(2, n):
    fibonacci.append(fibonacci[i-1] + fibonacci[i-2])

print("Fibonacci:", fibonacci)

```

Lernziele: Listen dynamisch erweitern, mathematische Folgen.

Lernreihenfolge (Fortsetzung)

- 21. Musikinstrument → class + __init__
- 22. Gitarre → Vererbung mit super()
- 23. GGT/KGV → Euclid-Algorithmus
- 24. Fibonacci → Listen + mathematische Folgen

Nächster Schritt: Datei-Generatoren + Erweiterte Algorithmen (Teil 7 - FINAL)

LF5-Grundlagen: Python-Übungen (Teil 7 - FINAL)

25. Gleichungs-Generator (Dateien schreiben)

Ziel: Automatische Dateierstellung mit Schleifen.

```

def main():
    limit = int(input("Geben Sie die Grenze ein: "))
    filecounter = 1
    equations_per_file = 500

```

```

equationcount = 0
writer = None

try:
    writer = open(f"output{filecounter}.txt", "w", encoding="utf-8")

    for i in range(1, limit + 1):
        for j in range(1, limit + 1):
            for k in range(1, limit + 1):
                # Gemischt-kubische Gleichungen generieren
                b1 = i + j + k
                b2 = -i - j - k
                d1 = i * j * k

                writer.write(f"Gleichung {b1}x + {d1} = 0\n")
                writer.write(f"Gleichung {b2}x + {d1} = 0\n")
                equationcount += 2

            # Neue Datei bei Limit
            if equationcount >= equations_per_file:
                writer.close()
                filecounter += 1
                writer = open(f"output{filecounter}.txt", "w",
encoding="utf-8")
                equationcount = 0

        print("Die Gleichungen wurden in mehrere Dateien aufgeteilt.")

except IOError as e:
    print(f"Ein Fehler ist aufgetreten: {e}")
finally:
    if writer:
        writer.close()

if __name__ == "__main__":
    main()

```

Lernziele: `if __name__ == "__main__"`, Datei-Rotation, nested Schleifen.

26. Quadratische Gleichungen lösen (Teiler-Methode)

Ziel: Mathematische Algorithmen mit Funktionen.

```

import math

def teiler_analyse(n):
    listet = []
    listek = []
    for i in range(1, int(math.sqrt(n)) + 1):
        if n % i == 0:

```

```

        teiler = i
        komplement = n // i
        listet.append(teiler)
        listek.append(komplement)
    return listet, listek

p = 45
q = -46
list_teiler, list_komp = teiler_analyse(abs(q))

print("Teileranalyse 241:", list_teiler_analyse(241))
print("Summenanalyse:", summen_analyse(p, q, [1], [2, 3, 12, 6, 4]))

```

Lernziele: Mathematische Problem-Lösung, Funktions-Ketten.

LF5-Kurs ABGESCHLOSSEN!

Komplette Lernreihenfolge (28 Übungen)

Teil 1: Noten → Passwort → Teiler → Begrüßung
 Teil 2: Schülernoten → Einkaufsliste → Rechner → Triangel
 Teil 3: Statistics → Kehrwert → Funktionen → Temperatur
 Teil 4: Dateien → Listen-Methoden → Quadrat → Backslash
 Teil 5: Zufallszahl → Primzahlen → Palindrom → Basis
 Teil 6: Musikinstrument → Gitarre → GGT → Fibonacci
 Teil 7: Gleichungs-Generator → Quadratische Gleichungen

** Erreichte Kompetenzen **

- **Grundlagen:** input/print/variablen
 - **Kontrollstrukturen:** if/elif/while/for/break
 - **Listen:** append/sort/remove/min/max/sum
 - **Funktionen:** def/return/parameter/try/except
 - **Dateien:** open/read/write/encoding
 - **OOP:** class/init/super/vererbung
 - **Algorithmen:** Primzahlen/Palindrome/Basis/GGT
-

LF5-Erweiterte: Mathe-Algorithmen (Teil 8)

27. Monte-Carlo Pi-Berechnung

Ziel: Zufällige Punkte → Kreisanteil → Pi approximieren.

```

import random
import math

def generate_point():
    """Generiert zufälligen Punkt im Quadrat [-1,1]x[-1,1]"""
    x = random.uniform(-1, 1)
    y = random.uniform(-1, 1)
    return x, y

def is_in_circle(x, y):
    """Prüft ob Punkt im Einheitskreis liegt"""
    return x**2 + y**2 <= 1

def monte_carlo_pi(num_points):
    """Berechnet Pi mit Monte-Carlo-Methode"""
    points_inside = 0
    for _ in range(num_points):
        x, y = generate_point()
        if is_in_circle(x, y):
            points_inside += 1
    pi_approx = 4 * points_inside / num_points
    return pi_approx

# Test mit verschiedenen Punktzahlen
print("Punkte\tPi-Approx\tAbweichung")
for num_points in [100, 1000, 10000]:
    pi_approx = monte_carlo_pi(num_points)
    abweichung = abs(pi_approx - math.pi)
    print(f"{num_points:}, }\t{pi_approx:.6f}\t{abweichung:.6f}")

```

Lernziele: Statistische Methode, $**2$ (Potenz), Flächenverhältnis.

28. Buchstaben-Zähler (a-z Häufigkeit)

Ziel: Dictionary für Buchstaben-Statistik.

```

x = input("Geben Sie ein Wort ein: ")
y = x.lower()
liste = "abcdefghijklmnopqrstuvwxyz"
count = {buchstabe: 0 for buchstabe in liste}

for i in range(len(liste)):
    for j in range(len(y)):
        if y[j] == liste[i]:
            count[liste[i]] += 1

print("Anzahl der Buchstaben:")
for i in range(26):
    if count[liste[i]] != 0:

```



```
print(f"{count[liste[i]]} x {liste[i]}")
print(f"Anzahl der Buchstaben insgesamt: {len(x)}")
```

Lernziele: Dictionary `{key: value}`, Nested Loops für Zählung.

29. Periodische Binärdarstellung (1/n)

Ziel: Periodische Brüche als Binärzahl (z.B. $1/3 = 0.01$ wiederholt).

```
def periodische_binaer_darstellung(n, max_stellen=50):
    """1/n als periodische Binärzahl"""
    nachkomma_teile = []
    reste = {}
    zahl = 1
    stelle = 0

    while zahl != 0 and stelle < max_stellen:
        if zahl in reste:
            start = reste[zahl]
            nicht_periodisch = "".join(nachkomma_teile[:start])
            periodisch = "".join(nachkomma_teile[start:])
            return f"0.{nicht_periodisch}({periodisch})"

        reste[zahl] = stelle
        zahl *= 2
        ziffer = zahl // n
        zahl %= n
        nachkomma_teile.append(str(ziffer))
        stelle += 1

    return "0." + "".join(nachkomma_teile)

# Test
for n in [3, 7, 11]:
    print(f"Binärdarstellung von 1/{n}: {periodische_binaer_darstellung(n)}")
```

Lernziele: Dictionary für Periodenerkennung, Rest-Rechnung.

30. Erweiterte Basis-Umrechnung (beliebige Basis)

Ziel: Dezimal → Basis 2-36 mit Großbuchstaben.

```
def dezimal_zu_basis(zahl, basis):
    """Dezimalzahl in Basis 2-36 umwandeln"""
    zeichen = "0123456789ABCDEFGHIJKLMNOPQRSTUVWXYZ"
    if zahl == 0:
        return "0"
```

```

basis_zahl = ""
while zahl > 0:
    rest = zahl % basis
    basis_zahl = zeichen[rest] + basis_zahl
    zahl = zahl // basis
return basis_zahl

# Interaktive Eingabe
b = int(input("Geben Sie die Basis 2-36 ein: "))
dezimal_zahl = int(input("Geben Sie eine Dezimalzahl ein: "))
print(f"Die Dezimalzahl {dezimal_zahl} zur Basis {b} ist:
{dezimal_zu_basis(dezimal_zahl, b)}")

```

Lernziele: Universelle Basis-Funktion, Zeichen-Array.

Lernreihenfolge (Erweiterte Mathe)

```

27. Monte-Carlo → Statistische Pi-Berechnung
28. Buchstaben-Zähler → Dictionary + Nested Loops
29. Periodische Binär → Rest-Algorithmus + Dict
30. Basis 2-36 → Universelle Umrechnung

```

Nächster Schritt: Datei-Generatoren + GUI (Teil 9)

LF5-Erweiterte: Datei-Generatoren (Teil 9)

31. Tetraeder-Koordinaten Generator

Ziel: 3D-Mathematik + Massendaten in Dateien.

```

def print_array(liste):
    """Array in Dateien schreiben mit Datei-Rotation"""
    counter = 0
    filecounter = 1

    try:
        writer = open(f"ausgabe{filecounter}.txt", "w", encoding="utf-8")

        for i in range(len(liste)):
            for j in range(i + 1, len(liste)):
                if liste[i][3] == liste[j][3]: # Gleiche Summe
                    content = f"tetraeder {liste[i]}, {liste[j]}\n"
                    writer.write(content)
                    counter += 1
    
```

```

# Datei-Rotation bei Limits
if counter >= 50000:
    writer.close()
    filecounter += 1
    writer = open(f"ausgabe{filecounter}.txt", "w",
encoding="utf-8")
    counter = 0

writer.close()
print(f"Neue Datei erstellt: ausgabe{filecounter}.txt")

except IOError as e:
    print(f"Fehler beim Schreiben in die Datei: {e}")

# Beispiel-Daten
triples = [[1,2,3,6], [4,5,6,15], [7,8,9,24]]
print_array(triples)

```

Lernziele: Datei-Rotation, nested Indizes [3], Massendaten.

32. Kubische Gleichungen Generator

Ziel: Mathematische Koeffizienten + Formatierung.

```

def format_coeff(coeff):
    """Formatierung mit Vorzeichen"""
    return str(coeff) if coeff >= 0 else f"{{coeff}}"

def generate_cubic_equations(limit):
    """Generiert gemischt-kubische Gleichungen"""
    filecounter = 1
    equations_per_file = 500
    equationcount = 0
    writer = None

    try:
        writer = open(f"output{filecounter}.txt", "w", encoding="utf-8")

        for i in range(1, limit + 1):
            for j in range(1, limit + 1):
                for k in range(1, limit + 1):
                    # Koeffizienten berechnen
                    b1 = i + j + k
                    b2 = -i - j - k
                    c1 = i * j + j * k + i * k
                    d1 = i * j * k

                    writer.write(f"Gleichung {format_coeff(b1)}x +
{format_coeff(c1)}x + {d1} = 0\n")
                    writer.write(f"Gleichung {format_coeff(b2)}x +

```

```

{format_coeff(c1)}x + {d1} = 0\n")
    equationcount += 2

    if equationcount >= equations_per_file:
        writer.close()
        filecounter += 1
        writer = open(f"output{filecounter}.txt", "w",
encoding="utf-8")
        equationcount = 0

    except IOError as e:
        print(f"Fehler: {e}")
    finally:
        if writer:
            writer.close()

generate_cubic_equations(20)

```

Lernziele: f-strings mit Variablen, komplexe Mathe-Formeln.

33. Schüler-Noten Random-Generator

Ziel: List-Iteration + Bedingte Ausgabe.

```

import random

schueler = ["Benny", "Patrick", "Holger", "Tibor", "Luke",
            "Alex", "Daniela", "Thomas", "Florian", "Daniel",
            "Marlon", "Roy", "David", "Gerardo", "Senad",
            "Liridona", "Houari", "Alexander", "Onur", "Fabio"]

for _ in range(3): # 3 Durchläufe
    for umschueler in schueler:
        note = random.randint(1, 6)
        if umschueler in ["Liridona", "Alexander"]:
            print(f"Umschüler {umschueler} bekommt die {note}!")
        else:
            print(f"Umschüler {umschueler} bekommt die {note}.")

```

Lernziele: in-Operator für Listen, for-else-Struktur.

34. Einkaufsliste Tabelle (Datei-Ausgabe)

Ziel: Formatierte Tabellen + with open().

```

table_output = []
einkaufsliste = []

```

```

# Produkte eingeben
while True:
    produkt = input("Gib das zu kaufende Produkt ein (beenden mit 'y'): ")
    if produkt == "y":
        break
    einkaufsliste.append(produkt)

# Einzigartige Produkte + Zählung
produkt_count = {}
for produkt in einkaufsliste:
    produkt_count[produkt] = produkt_count.get(produkt, 0) + 1

# Tabelle erstellen
maxlength = max(len(word) for word in einkaufsliste) if einkaufsliste else 7
table_output.append("-" * (maxlength + 3) + "-----")
table_output.append(f"Produkt{' ' * (maxlength - 7)}Anzahl")
table_output.append("-" * (maxlength + 3) + "-----")

for produkt, count in produkt_count.items():
    table_output.append(f"{produkt:<{maxlength}} x{count}")

table_output.append("-" * (maxlength + 3) + "-----")

# In Datei schreiben
with open("tabelle.txt", "w", encoding="utf-8") as file:
    for line in table_output:
        file.write(line + "\n")

print("Die Tabelle wurde in tabelle.txt gespeichert.")

```

Lernziele: `with open()` (automatisches Schließen), `<{n}` Formatierung, `dict.get()`.

Lernreihenfolge (Generatoren)

31. Tetraeder → 3D-Daten + Datei-Rotation
32. Kubische Gleichungen → Mathe-Koeffizienten
33. Schüler-Noten → Random + Bedingungen
34. Tabelle → Professionelle Datei-Ausgabe

Nächster Schritt: GUI + tkinter (Teil 10 - FINAL)

LF5-Rest: Alle anderen Programme (Referenz-Liste)

WIEDERHOLUNGEN (überspringen – Konzepte bekannt)

1. Notenberechnung x3 Varianten (if/elif)
2. Passwortprüfung x2 (while + break)
3. Schülernoten x4 (mit/ohne statistics)
4. Einkaufsliste x3 (Duplikate zählen)
5. Rechner x2 (Funktionen + try/except)
6. Teiler-Funktion x2 (math.sqrt)
7. Sternchen-Triangeln x3 (range Variationen)

NICHT WICHTIG (Test-Code/Fehlerhaft)

1. Backslash-Ausgabe (`\\` → `\`)
2. "Hello Roy!" / "Wer dies liest, ist doof"
3. Random Präsentation (Roy/Gerardo/Benny)
4. Listen-Index-Beispiele (`zahlen[0]`/`zahlen[5]`)
5. Einfache Addition (`5+9=14`)

FORTGESCHRITTEN (Bonus – nach LF5)

1. ****Pythagoras-Generator**** (primitive Tripel $a^2+b^2=c^2$)
 - ggT-Funktion + m,n-Iteration
 - CSV-Ausgabe + matplotlib Histogramm
2. ****Cardano-Formel**** (kubische Gleichungen $ax^3+bx^2+cx+d=0$)
 - Reelle Lösungen berechnen
3. ****Tetraeder/Oktaeder-Formeln**** (3D-Koordinaten)
 - Komplexe algebraische Ausdrücke
4. ****Musterlösung-Generator****
 - `output1.txt` → `musterloesung4.txt` (alle Schritte)
5. ****Regex-Gleichungs-Parser****
 - Dateien durchsuchen (`rx±x±0`)

Dateistruktur-Empfehlung

```

LF5-Komplett/
├── 01-Grundlagen/          # Teil 1-7 (WICHTIG!)
├── 02-Erweiterte/          # Teil 8-9 (SEHR GUT!)
├── 03-Optional/
│   ├── GUI/                # tkinter-Programme
│   │   ├── QuadraticEquationGUI.py
│   │   └── output1.txt
│   └── Bonus/
│       └── Pythagoras.py

```

```
|      |
|      |— Cardano.py
|      |— 3D-Tetraeder.py
|— Rest/      # DIESER ORDNERT
|   |— Referenz-Liste.md # ← DU BIST HIER!
```

CHECKLISTE – DU HAST FERTIG:

Grundlagen (28 Übungen): input/if/listen/funktionen
Erweiterte (8 Übungen): Pi/Buchstaben/Basis/Generatoren
Rest (27 Programme): Dokumentiert & archiviert
Gesamt: 63 Programme aus LF5 gemeistert!

→ LF5 100% ABGESCHLOSSEN! Alle Programme dokumentiert und sortiert.

[

LF5-Spezial: Einkaufslisten, Sternchen & Listen-Indizes

EINKAUFSLISTEN (3 Varianten)

Variante 1: Basis (fertig → stop)

```
einkaufsliste = []
artikel = input("tippe 'fertig' um zu beenden: ")
while artikel != "fertig":
    einkaufsliste.append(artikel)
    artikel = input("tippe 'fertig' um zu beenden: ")
einkaufsliste.remove("fertig")
print(einkaufsliste)
```

Unterschied: Einfachste Version, manuell "fertig" entfernen.

Variante 2: Einzigartige + Zählung

```
einzigartige_liste = []
for element in einkaufsliste:
    if element not in einzigartige_liste:
        einzigartige_liste.append(element)

for produkt in einzigartige_liste:
    count = einkaufsliste.count(produkt) # Einfacher mit .count()!
    print(f"{produkt}: x{count}")
```

Variante 3: Professionelle Tabelle

```

maxlength = max(len(w) for w in einkaufsliste)
print("-" * (maxlength + 10))
print(f"{'Produkt':<{maxlength}} Anzahl")
print("-" * (maxlength + 10))
for produkt in einzigartige_liste:
    print(f"{produkt:<{maxlength}} x{einkaufsliste.count(produkt)}")

```

STERNCHEN-TRIANGELN (4 Varianten)

1. Aufbau (1,3,5... Sternchen)

```

anzahl = 5
for i in range(1, anzahl + 1):
    print(" " * (anzahl - i) + "*" * (2 * i - 1))

```

```

    *
   **
  ***
 ****
*****

```

2. Abbau (umgekehrt)

```

for i in range(anzahl - 1, 0, -1):
    print(" " * (anzahl - i) + "*" * (2 * i - 1))

```

```

*****
****
***
**
*

```

3. Vollständiger Diamant

```

# Auf + Abbau kombinieren
for i in range(1, anzahl + 1):    # Auf
for i in range(anzahl - 1, 0, -1): # Ab

```


4. Einfaches Quadrat

```
for i in range(anzahl):  
    print("*" * anzahl)
```

```
*****  
*****  
*****  
*****
```

LISTEN-INDIZES (Wichtig!)

```
zahlen = [3, 7, 2, 9, 4, 1]  
  
print(zahlen[0])      # 3      ← ERSTES Element  
print(zahlen[5])      # 1      ← LETZTES Element  
print(zahlen[-1])     # 1      ← LETZTES (negativ)  
print(zahlen[-2])     # 4      ← VORLETZTES  
  
zahlen.sort()         # [1,2,3,4,7,9]  
print(zahlen[0])      # 1      ← Minimum nach sort  
print(zahlen[-1])     # 9      ← Maximum nach sort
```

Trick: Nach `sort()` ist `liste[0]` immer Minimum, `liste[-1]` immer Maximum!

SCHNELL-ÜBERSICHT

EINKAUFSLISTEN:

- V1: while + append + remove("fertig")
- V2: .count() für Duplikate
- V3: :<n Formatierung für Tabellen

STERNCHEN:

- Auf: range(1, anzahl+1), 2*i-1 Sterne
- Ab: range(anzahl-1, 0, -1)
- Diamant: Auf + Ab

INDIZES:

- zahlen[0] → erstes
- zahlen[-1] → letztes
- zahlen.sort() → min[0]/max[-1]